SOC_25 – Final Report

RL: Race and Learning

Siddharth
Roll No. 23B0378
IIT Bombay

Abstract

This project trains a reinforcement learning agent to drive in the `CarRacing-v3` environment of Gymnasium directly from pixels. The agent is a convolutional actor–critic network trained with Proximal Policy Optimization (PPO) and Generalized Advantage Estimation (GAE). Observations are a stack of four grayscale frames, the continuous control space is replaced by five discrete actions, and the reward is lightly shaped to discourage leaving the road. After roughly 117k agent steps, the best checkpoint reaches a mean score of 421.9 over 20 evaluation laps when actions are sampled from the policy. Always choosing the most likely action performs far worse, which is discussed in the results.

1 Introduction

`CarRacing-v3` is a top-down racing task. The agent sees a 96×96 RGB image and controls steering, gas and brake. It earns $+1000/N$ for each of the N track tiles visited and -0.1 per frame, and the episode ends if the car leaves the playfield (a -100 penalty). Learning from pixels with no hand-crafted state makes it a good test of end-to-end deep reinforcement learning.

The goals of this project were:

- build a pixel-based PPO agent from scratch in PyTorch;
- make the car drive faster and take corners rather than going straight;
- evaluate the trained policy properly, not on a single lap.

2 Environment and Preprocessing

2.1 Observation

Each frame is converted to grayscale with OpenCV. The last four frames are stacked, so the network input has shape $4 \times 96 \times 96$ and can infer the car's motion from differences between frames. At reset, the first 50 frames (the zoom-in intro) are skipped.

2.2 Action repeat

Every chosen action is applied for 4 consecutive game frames. One *agent step* therefore corresponds to 4 frames, which shortens the effective horizon and speeds up learning.

2.3 Discrete action space

The policy outputs a categorical distribution over five actions, each mapped to a continuous $(\text{steer}, \text{gas}, \text{brake})$ vector (Table 1). Steering is combined with gas so that the car does not lose speed while turning.

Index	Meaning	Steer	Gas	Brake
0	gas	0.0	1.0	0.0
1	left + gas	-1.0	0.5	0.0
2	right + gas	1.0	0.5	0.0
3	brake	0.0	0.0	0.5
4	coast	0.0	0.0	0.0

Table 1: Mapping from the five discrete actions to continuous controls.

2.4 Reward shaping

Let r_{raw} be the game reward summed over the 4 repeated frames. The training reward is

$$r = \text{clip}\big(0.25\,(r_{\text{raw}} - 1 \cdot \mathbb{1}[\text{off-track}]),\, -2,\, 2\big),$$

where *off-track* means none of the four wheels touches a road tile. If the car makes no positive progress for 50 consecutive agent steps, the episode is terminated early with an extra -1 penalty. The raw game score is kept separately and used for all logging and evaluation.

3 Model

3.1 Architecture

The network has a shared convolutional trunk followed by separate actor and critic heads (Table 2). Pixels are scaled to $[0, 1]$ before the first layer. The flattened convolutional output has $64 \times 8 \times 8 = 4096$ features.

Block	Layer
Conv 1	Conv2d, 4→32, kernel 8×8, stride 4, ReLU
Conv 2	Conv2d, 32→64, kernel 4×4, stride 2, ReLU
Conv 3	Conv2d, 64→64, kernel 3×3, stride 1, ReLU
Actor	Linear 4096→256, ReLU, Linear 256→5 (logits)
Critic	Linear 4096→256, ReLU, Linear 256→1 (value)

Table 2: Actor–critic network.

All layers use orthogonal initialization (gain $\sqrt{2}$), except the final actor layer (gain 0.01, so the initial policy is nearly uniform) and the final critic layer (gain 1.0).

4 Training Algorithm

The agent is trained with PPO using the clipped surrogate objective. With probability ratio $\rho_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ and advantage $\hat{A}_t$,

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\big(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\hat{A}_t\big)\Big].$$

The total loss is

$$L = -L^{\text{CLIP}} + c_v\, \mathbb{E}_t\big[(V_\theta(s_t) - \hat{R}_t)^2\big] - c_e\, \mathbb{E}_t\big[H(\pi_\theta(\cdot|s_t))\big].$$

Advantages use GAE,

$$\delta_t = r_t + \gamma V(s_{t+1})(1 - d_t) - V(s_t), \qquad \hat{A}_t = \delta_t + \gamma\lambda(1 - d_t)\hat{A}_{t+1},$$

with the critic's value of the last state used as the bootstrap value, and $\hat{R}_t = \hat{A}_t + V(s_t)$.

Each update collects 2048 agent steps, normalizes the advantages, and performs 4 epochs of minibatch updates (minibatch size 256). Gradients are clipped to a norm of 0.5 and the learning rate decays linearly toward zero over the planned run. The best-scoring policy (by the mean raw score of the last 10 training episodes) is saved as a checkpoint.

Parameter	Value	Parameter	Value
Planned agent steps	300,000	Rollout length	2048
PPO epochs	4	Minibatch size	256
Learning rate (Adam)	2.5×10^{-4}	Adam ϵ	10^{-5}
Discount γ	0.99	GAE λ	0.95
Clip ϵ	0.2	Value coef. c_v	0.5
Entropy coef. c_e	0.01	Early-stop patience	50 agent steps
Action repeat	4 frames	Frame stack	4

Table 3: Hyperparameters.

5 Results

5.1 Training progress

Each update took about 90 seconds, because most of the time goes to the CPU-based Box2D simulation. Training was stopped manually after update 57 (about 117k agent steps) once the average score had plateaued around 350–400 with large fluctuations. Figure 1 shows the average raw score over the last 10 training episodes at selected updates, taken from the training log.

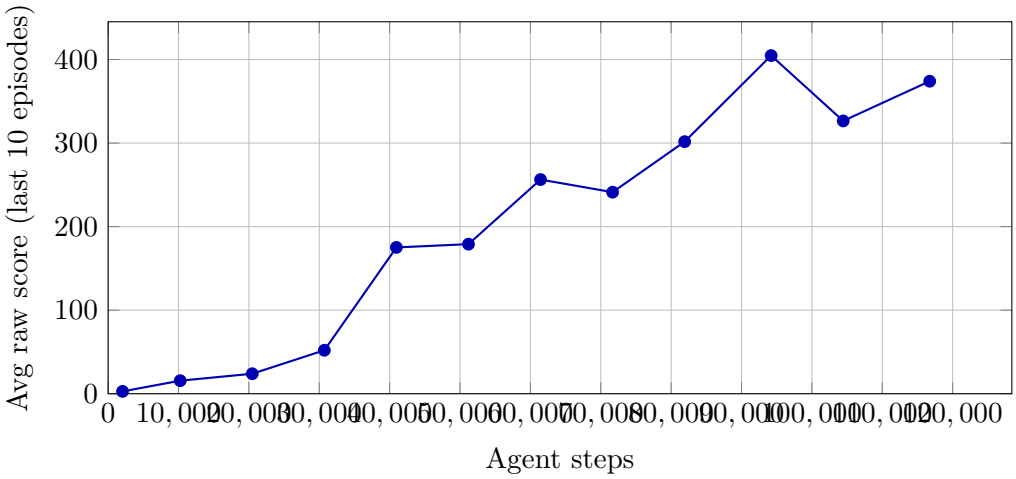


Figure 1: Training progress (selected points from the training log).

5.2 Evaluation

The best checkpoint was evaluated on 20 laps with seeds 2000–2019 (not used in training), with a cap of 2000 game frames per lap and no early termination. Actions were sampled from the policy's output distribution. The scores were:

$$489,\ 514,\ 231,\ 233,\ 98,\ 777,\ 326,\ 443,\ 505,\ 378,$$
$$173,\ 169,\ 468,\ 779,\ 112,\ 667,\ 758,\ 788,\ 602,\ -72.$$

Statistic (20 sampled laps)	Score
Mean	421.9
Standard deviation	251.9
Minimum	−72
Maximum	788

Table 4: Evaluation of the best checkpoint.

5.3 Sampled versus greedy actions

On 5 laps per mode, always taking the most likely action scored $[-28, -105, -90, -105, -6]$ (mean -66.6), whereas sampling scored $[16, 186, 347, 762, 292]$ (mean 320.3). The policy learned to steer by mixing actions, and with only five actions the single most likely choice often sends the car off the road. All reported results therefore use sampled actions. The 5-lap comparison is a small sample and is meant only to show the gap.

5.4 Recorded video

A single evaluation lap is recorded offscreen with Gymnasium's `RecordVideo` wrapper and embedded in the notebook as a base64 HTML5 video. The recorded lap with sampled actions scored 152.8, which is one draw from a wide score distribution.

6 Challenges and Fixes

- **Original PPO update was incorrect.** It reused only the last observation and action of an episode. The final version stores the full rollout and updates on shuffled minibatches.
- **Slow car, no corners.** Actions that never combined steering with gas made the car slow down in turns, and 3 training episodes were far too few. Steer+gas actions, action repeat, longer rollouts and reward shaping addressed this.
- **Three videos.** Recording every training episode produced multiple clips; the final version records one evaluation lap after training.
- **Stale notebook state.** Re-running cells caused signature mismatches and empty video folders until the session was restarted.

7 Limitations and Future Work

- The result varies a lot between laps (standard deviation 251.9, worst lap -72), and the task is usually considered solved at a score around 900, so the agent is not at that level.
- Training used one run and one seed and stopped at about 117k steps. The best checkpoint was chosen using a noisy 10-episode average.
- The reward shaping and action repeat were not ablated, so their individual effect is unmeasured.
- Episodes that end by time-limit truncation are treated like terminal states when computing advantages, which slightly biases the value targets.
- Possible improvements: longer training, multiple seeds, a continuous action policy, vectorized environments for faster rollouts, and ablations of each design choice.

8 Conclusion

A pixel-based PPO agent with a 4-frame grayscale stack, five steer/gas/brake actions and light reward shaping learns to follow the road and take corners in `CarRacing-v3`, reaching a mean score of 421.9 over 20 held-out laps with sampled actions. The agent is inconsistent and falls well short of expert performance, but the pipeline, evaluation and fixes above give a working base for longer training and further experiments.